

Deployment Guide

Cebuana Lhuillier Parking Reservation System

Version: 2.0 | Last Updated: February 24, 2026

Target: Amazon Web Services (AWS)

1. Architecture Overview

The application consists of three main components that need to be deployed:

```
[Route 53 / Custom Domain]
|
[Application Load Balancer (ALB)]
├── /api/* → Backend Container (FastAPI :8001)
└── /*    → Frontend Container (React :3000)
|
[Amazon DocumentDB / MongoDB Atlas]
|
[EBS / EFS Volume for file uploads]
```

2. Prerequisites

2.1 AWS Account & Services

Service	Purpose	Required
AWS Account	Cloud hosting	Yes
IAM	Access management	Yes
ECS (Fargate) or EKS	Container orchestration	Yes
ECR	Container image registry	Yes
ALB	Load balancer with path-based routing	Yes
Route 53	DNS management	Optional
ACM	SSL/TLS certificate	Yes (for HTTPS)
DocumentDB or MongoDB Atlas	Database	Yes

Service	Purpose	Required
EBS/EFS	Persistent file storage	Yes
CloudWatch	Logging and monitoring	Recommended
Secrets Manager	Credential storage	Recommended

2.2 Tools Required

- AWS CLI (v2+)
- Docker (v20+)
- Node.js 18+ and Yarn
- Python 3.11+
- Git

2.3 Source Code

Ensure you have the latest codebase from the Git repository:

```
git clone <repository-url>
cd parking-reservation-system
```

3. Environment Configuration

3.1 Backend Environment Variables

Create `/app/backend/.env`:

```
MONGO_URL=mongodb://<username>:<password>@<documentdb-endpoint>:27017/<db-name>?retryWrites=f
DB_NAME=parking_reservation_db
JWT_SECRET=<generate-a-strong-random-string-64-chars>
JWT_EXPIRATION_HOURS=0.5
CORS_ORIGINS=https://your-domain.com,https://www.your-domain.com
EMERGENT_LLM_KEY=<your-emergent-llm-key>
```

Security: Never commit `.env` files to Git. Use AWS Secrets Manager or ECS task definition secrets for production.

Variable Reference:

Variable	Description	Example
<code>MONGO_URL</code>	MongoDB/DocumentDB connection string	<code>mongodb://user:pass@host:27017/db</code>
<code>DB_NAME</code>	Database name	<code>parking_reservation_db</code>
<code>JWT_SECRET</code>	Secret for JWT signing (must be unique, strong)	Random 64-character string
<code>JWT_EXPIRATION_HOURS</code>	Session duration in hours	<code>0.5</code> (30 minutes)
<code>CORS_ORIGINS</code>	Comma-separated allowed origins	<code>https://parking.yourcompany.com</code>
<code>EMERGENT_LLM_KEY</code>	API key for AI insights (optional)	Key from Emergent LLM

3.2 Frontend Environment Variables

Create `/app/frontend/.env` :

```
REACT_APP_BACKEND_URL=https://your-domain.com
```

The `REACT_APP_BACKEND_URL` is baked into the React build at compile time. It must point to the public URL of your deployment.

4. Database Setup

Option A: Amazon DocumentDB (Recommended for AWS)

1. **Create a DocumentDB cluster** in your VPC:

```

AWS Console → DocumentDB → Create Cluster

```
2. Engine: MongoDB 5.0 compatible
3. Instance class: db.r6g.large (or smaller for testing)
4. Number of instances: 2 (primary + replica)
5. VPC: Same as your ECS/EKS cluster
6. Encryption: Enabled

```


```

7. **Download the CA certificate** for TLS connections:

```
bash wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

8. **Create the database and indexes:**

The application automatically creates indexes on startup, but you can verify:

```
bash mongosh "mongodb://<user>:<pass>@<endpoint>:27017/parking_reservation_db?
tls=true&tlsCAFile=global-bundle.pem"
```

Option B: MongoDB Atlas

1. Create a MongoDB Atlas cluster (M10+ recommended).
2. Whitelist your ECS/EKS cluster's NAT Gateway IPs.
3. Use the Atlas connection string in `MONGO_URL`.

Database Indexes

The application creates 25+ indexes automatically on startup. No manual index creation is required.

5. Building Docker Images

5.1 Backend Dockerfile

Create `backend/Dockerfile`:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p /app/uploads

EXPOSE 8001

CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8001", "--workers", "4"]
```

5.2 Frontend Dockerfile

Create `frontend/Dockerfile`:

```
FROM node:18-alpine AS build

WORKDIR /app

COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile

COPY . .
RUN yarn build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 3000

CMD ["nginx", "-g", "daemon off;"]
```

5.3 Frontend Nginx Config

Create `frontend/nginx.conf`:

```
server {
    listen 3000;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /health {
        return 200 'ok';
        add_header Content-Type text/plain;
    }
}
```

5.4 Build and Push Images

```
# Authenticate with ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin .

# Create ECR repositories
aws ecr create-repository --repository-name parking-backend
aws ecr create-repository --repository-name parking-frontend

# Build and push backend
cd backend
docker build -t parking-backend .
docker tag parking-backend:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/parking-backend:latest
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/parking-backend:latest

# Build and push frontend
cd ../frontend
docker build -t parking-frontend --build-arg REACT_APP_BACKEND_URL=https://your-domain.com .
docker tag parking-frontend:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/parking-frontend:latest
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/parking-frontend:latest
```

6. ECS Deployment (Fargate)

6.1 Create ECS Cluster

```
AWS Console → ECS → Create Cluster
- Cluster name: parking-cluster
- Infrastructure: AWS Fargate
```

6.2 Create Task Definition

```

{
  "family": "parking-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "executionRoleArn": "arn:aws:iam::<account-id>:role/ecsTaskExecutionRole",
  "containerDefinitions": [
    {
      "name": "backend",
      "image": "<account-id>.dkr.ecr.us-east-1.amazonaws.com/parking-backend:latest",
      "portMappings": [{"containerPort": 8001, "protocol": "tcp"}],
      "environment": [
        {"name": "DB_NAME", "value": "parking_reservation_db"},
        {"name": "JWT_EXPIRATION_HOURS", "value": "0.5"},
        {"name": "CORS_ORIGINS", "value": "https://your-domain.com"}
      ],
      "secrets": [
        {"name": "MONGO_URL", "valueFrom": "arn:aws:secretsmanager:us-east-1:<account-id>:secret/MONGO_URL"},
        {"name": "JWT_SECRET", "valueFrom": "arn:aws:secretsmanager:us-east-1:<account-id>:secret/JWT_SECRET"},
        {"name": "EMERGENT_LLM_KEY", "valueFrom": "arn:aws:secretsmanager:us-east-1:<account-id>:secret/EMERGENT_LLM_KEY"}
      ],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:8001/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/parking-backend",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "ecs"
        }
      }
    },
    {
      "name": "frontend",
      "image": "<account-id>.dkr.ecr.us-east-1.amazonaws.com/parking-frontend:latest",
      "portMappings": [{"containerPort": 3000, "protocol": "tcp"}],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:3000/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/parking-frontend",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "ecs"
        }
      }
    }
  ]
}

```

```

    }
  }
]
}

```

6.3 Create ECS Service

```

aws ecs create-service \
  --cluster parking-cluster \
  --service-name parking-service \
  --task-definition parking-app \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsVpcConfiguration={subnets=[subnet-xxx,subnet-yyy],securityGroups=[sg-xxx,sg-yyy]}" \
  --load-balancers "targetGroupArn=arn:aws:elasticloadbalancing:...:targetgroup/parking-backend" \
  --load-balancers "targetGroupArn=arn:aws:elasticloadbalancing:...:targetgroup/parking-frontend"

```

7. Load Balancer Configuration

7.1 Create Application Load Balancer

AWS Console → EC2 → Load Balancers → Create ALB

- Name: parking-alb
- Scheme: Internet-facing
- Listeners: HTTPS :443
- VPC: Same as ECS cluster
- Security Group: Allow inbound 443

7.2 Target Groups

Create two target groups:

Target Group	Port	Health Check	Path Pattern
parking-backend	8001	GET /health	/api/*
parking-frontend	3000	GET /health	/* (default)

7.3 Listener Rules

Configure HTTPS :443 listener rules:

Priority	Condition	Action
1	Path pattern: <code>/api/*</code>	Forward to <code>parking-backend</code> target group
2	Default	Forward to <code>parking-frontend</code> target group

7.4 SSL Certificate

1. Request a certificate from ACM for your domain.
2. Validate via DNS (add CNAME record).
3. Attach to the ALB HTTPS listener.

8. DNS Configuration

Using Route 53

1. Create a hosted zone for your domain.
2. Create an **A record** (Alias) pointing to the ALB.

`parking.yourcompany.com` → ALB DNS name (alias)

9. Persistent Storage

File Uploads

Floor plan images are stored in `/app/uploads/`. For production:

Option A: EFS (Elastic File System)

1. Create an EFS file system in the same VPC.
2. Mount it to the backend container at `/app/uploads`.
3. Add to the task definition:

```
json "volumes": [{"name": "uploads", "efsVolumeConfiguration": {"fileSystemId": "fs-xxx"}}, {"mountPoints": [{"sourceVolume": "uploads", "containerPath": "/app/uploads"}]}
```

Option B: S3 (Recommended for scale)

- Modify the backend to upload images to S3 instead of local disk.
- Serve images via CloudFront CDN.

10. Monitoring & Logging

10.1 CloudWatch Logs

Both containers are configured to send logs to CloudWatch:

- Backend: `/ecs/parking-backend`
- Frontend: `/ecs/parking-frontend`

10.2 Health Checks

- Backend: `GET /health` → returns `{"status": "healthy"}`
- Frontend: `GET /health` → returns `ok`

10.3 CloudWatch Alarms (Recommended)

Alarm	Metric	Threshold
High CPU	ECS CPU utilization	> 80% for 5 min
High Memory	ECS memory utilization	> 85% for 5 min
5xx Errors	ALB 5xx count	> 10 per 5 min
Unhealthy Targets	ALB healthy host count	< desired count

10.4 Application-Level Logging

The backend uses Python's `logging` module. Key log sources:

- `parking_app` — Main application logger
- Background task logs:
 - `auto_mark_no_shows()` — Runs every 5 minutes, marks expired reservations as no-show and releases slots based on `no_show_at` timestamp
 - `check_waitlist_expiry()` — Runs every 60 seconds, expires waitlist notifications past the configured window and notifies next user
- Error logs for database operations

11. Security Checklist

Item	Action	Status
HTTPS only	Redirect HTTP to HTTPS on ALB	Required
JWT secret	Generate a unique 64-character random string	Required

Item	Action	Status
Database credentials	Store in AWS Secrets Manager	Required
Security headers	Auto-applied by middleware (CSP, HSTS, X-Frame-Options)	Built-in
Network isolation	Backend and DB in private subnets	Required
Rate limiting	5 requests/min on login endpoint	Built-in
CORS	Set <code>CORS_ORIGINS</code> to your domain only (not <code>*</code>)	Required
File upload limits	10MB max, image types only	Built-in
Session security	HttpOnly, Secure, SameSite cookies	Built-in

12. Deployment Verification Checklist

After deployment, verify each step:

```
# 1. Health check
curl https://your-domain.com/health
# Expected: {"status": "healthy"}

# 2. API root
curl https://your-domain.com/api/
# Expected: {"message": "Cebuana Lhuillier Parking API", "version": "2.0.0"}

# 3. Login test
curl -X POST https://your-domain.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "admin.test@cebuana.com", "password": "Test123!"}'
# Expected: 200 with user object

# 4. Frontend loads
curl -s https://your-domain.com | head -20
# Expected: HTML with React app

# 5. Check security headers
curl -I https://your-domain.com/api/
# Expected: X-Content-Type-Options, X-Frame-Options, CSP headers present

# 6. Database indexes
# Check backend logs for "Database indexes ensured" message
```

13. Scaling Considerations

Component	Scaling Strategy
Frontend	Horizontal (increase ECS task count)
Backend	Horizontal (increase ECS task count, stateless design)
Database	Vertical (upgrade DocumentDB instance class) or horizontal (add read replicas)
File Storage	Move to S3 + CloudFront for CDN

Auto-Scaling Rules (Recommended)

ECS Service Auto-Scaling:

- Target: CPU utilization at 70%
- Min tasks: 2
- Max tasks: 10
- Scale-in cooldown: 300 seconds
- Scale-out cooldown: 60 seconds

14. Backup & Recovery

Component	Strategy	Frequency
Database	DocumentDB automated backups	Daily (35-day retention)
File uploads	EFS backup or S3 versioning	Daily
Configuration	Infrastructure-as-Code (Terraform/CloudFormation)	Version controlled
Application	Container images in ECR	Tagged per deployment

15. Troubleshooting

Issue	Check
502 Bad Gateway	ECS tasks running? Health checks passing? Check CloudWatch logs.
Database connection error	Is the DocumentDB cluster in the same VPC? Security group allows : 27017?
Login fails	

Issue	Check
	Check JWT_SECRET is set. Verify admin user was seeded (check backend startup logs).
Images not loading	Check file upload directory is mounted (EFS). Verify <code>/api/uploads/</code> routing.
CORS errors	Ensure <code>CORS_ORIGINS</code> includes your exact domain (with <code>https://</code>).
Session issues	Verify cookie domain settings. Ensure HTTPS is enforced (secure cookie flag).

Related: [System Design](#) | [Test Plan](#)